

SOFTWARE REQUIREMENTS SPECIFICATION

Version 2.0

University of Hyderabad *Placement Management System*

□ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □

"That which liberates is knowledge"

Prepared by:

Team 22MCCE13 & 22MCCE16

School of Computer & Information Sciences

Reviewed in accordance with:

IEEE/ISO/IEC 29148:2018 • Software Engineering (Prof. Salman A. Moiz, AICTE, 2024)

Document date: April 2026

Document Control

Revision History

Version	Date	Author	Summary of Changes
1.0	Mar 2026	Team 22MCCE13, 22MCCE16	Initial SRS draft (IEEE 29148 structure).
2.0	Apr 2026	Team 22MCCE13, 22MCCE16	Reconciled with Sprint 1 implementation. Firestore committed as sole datastore (PostgreSQL removed). UoH-specific role model locked in. Self-Declaration form incorporated. Real UoH department list (12 schools, 42 departments). Reference list refreshed.

Approvals

This SRS is the baseline against which the design, implementation, and testing of the UoH Placement Management System will be verified. Any change after baseline requires explicit approval captured in the revision history.

Role	Name	Date
Student Author	22MCCE13	_____
Student Author	22MCCE16	_____
Project Supervisor	(SCIS Faculty)	_____
Course Instructor	Prof. Salman Abdul Moiz	_____

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the functional and non-functional requirements for the University of Hyderabad Placement Management System (UoH PMS), a web-based portal that centralises and digitises the campus placement process currently coordinated by the Placement Guidance and Advisory Bureau (PGAB) and individual school-level placement coordinators.

The document is the single source of truth from which the design specification, implementation, test plan, and acceptance criteria are derived. It is written to serve four audiences: the student development team, the academic supervisor and course instructor, future maintainers of the system, and reviewers evaluating the project against the course syllabus (Software Engineering, Prof. Salman A. Moiz, AICTE 2024).

Every requirement in this document is stated in a form that is unambiguous, verifiable, and free of implementation detail beyond what is necessary to pin down observable behaviour. The SRS follows IEEE/ISO/IEC 29148:2018 structure and uses the terminology established in Unit 2 of the prescribed textbook.

1.2 Scope

The UoH PMS replaces the current fragmented placement workflow — a mix of email threads, WhatsApp groups, shared spreadsheets, and ad-hoc forms — with an integrated role-based web application serving all twelve Schools of the University.

1.2.1 What the system will do

- Provide authenticated, role-aware accounts for Students, Faculty (Placement Coordinators), the central Training & Placement Officer (TPO), company Recruiters, and a System Administrator.
- Allow Students with a valid @uohyd.ac.in email to self-register, verify their identity through an email link plus a six-digit numeric OTP, and capture their self-declaration in compliance with the PGAB candidate declaration policy.
- Allow Companies to self-register and await administrative approval before posting opportunities.
- Provision Faculty and TPO accounts by administrative action only; Administrator accounts are seeded at installation and cannot be self-registered.
- Enable Companies, once approved, to post jobs and internships with explicit eligibility rules (School, Department, minimum CGPA, backlog limits, graduation year).
- Enable Students to browse, filter, and apply to openings for which they are eligible, with one-click application using their profile and uploaded resume.

- Enable Faculty to review their school's applicants, endorse candidates, and post school-level notices.
- Enable the TPO to approve companies, approve job postings, schedule placement drives, maintain the candidate blacklist per PGAB rules, and generate university-wide analytics and reports.
- Send transactional email notifications for registration, approval, application status changes, and scheduled interviews.
- Maintain an immutable audit log of security-sensitive actions (approvals, blacklist entries, role changes).

1.2.2 What the system will not do

- It does not integrate with any existing UoH academic records system (ERP, examination branch, or Ernet email directory); initial academic details are self-reported by students and verified by Faculty during endorsement.
- It does not process stipends, offer-letter dispatch, or payroll; those remain manual administrative activities.
- It does not conduct online proctored assessments (though a minimal MCQ quiz designer is a planned Sprint 4 extension and is explicitly out of the MVP scope).
- It does not offer a native mobile application at launch; the responsive web frontend is the only client.
- It does not replace the PGAB's policy-making function — it enforces existing PGAB rules but does not define new ones.

1.2.3 Benefits

The system reduces coordination overhead for the TPO and Placement Coordinators, gives Students a transparent view of every eligible opportunity and their application status, gives Companies a structured channel to reach the university's talent pool, and produces the placement statistics required for NIRF and IoE reporting as a by-product of normal operation rather than a separate manual exercise.

1.3 Definitions, Acronyms and Abbreviations

Term	Meaning
PMS	Placement Management System — the subject of this SRS.
UoH	University of Hyderabad — a central university and Institution of Eminence located in Gachibowli, Hyderabad.
PGAB	Placement Guidance and Advisory Bureau — the university-level body that oversees placement policy.
SCIS	School of Computer & Information Sciences.

Term	Meaning
SMS	School of Management Studies.
TPO	Training & Placement Officer — single university-level officer who administers the PMS and owns placement operations across all schools.
Faculty (in this SRS)	A faculty member designated as the Placement Coordinator of one school; each school has one Faculty account.
Recruiter / Company	An external representative of a hiring organisation who uses the PMS to post jobs.
CGPA	Cumulative Grade Point Average, on a 10-point scale.
RBAC	Role-Based Access Control.
OTP	One-Time Password — a six-digit numeric code delivered by email.
SRS	Software Requirements Specification.
FR / NFR	Functional Requirement / Non-Functional Requirement.
JWT	JSON Web Token — session credential issued by Firebase Authentication.
Firestore	Cloud Firestore, the NoSQL document database used by this system.
Self-Declaration	The PGAB-mandated candidate declaration (policy version 2024-26) that a Student must electronically sign before applying to any opportunity.
Blacklist	A record barring a Student from a configured number of subsequent drives, applied per PGAB rules (e.g., absenteeism, offer rejection after acceptance).
Audit Log	Append-only record of security-sensitive state changes; used for accountability and post-incident review.
IEEE 29148	The ISO/IEC/IEEE standard for requirements engineering (2018 edition) whose structure this SRS follows.

1.4 References

The following documents are the primary references for this SRS. Citations in the body of this document refer to these items.

1. Moiz, S. A. (2024). Software Engineering. New Delhi: All India Council for Technical Education (AICTE). ISBN 978-93-6027-556-3. (Primary textbook for the course; Units 2, 3, 4, and 5 govern the requirements, design, testing, and project-management aspects of this project respectively.)
2. ISO/IEC/IEEE 29148:2018 — Systems and software engineering — Life cycle processes — Requirements engineering. Geneva: ISO. (Structural standard followed by this SRS.)
3. University of Hyderabad. (2024). PGAB Candidate Self-Declaration Form, Session 2024–26. Placement Guidance and Advisory Bureau, UoH. (Policy source for the Self-Declaration feature and the blacklisting rules in §3.3.)

4. University of Hyderabad. Schools & Departments. Retrieved April 2026 from <https://uohyd.ac.in/schools-departments/> (Source of the authoritative 12-school, 42-department taxonomy used in the data model.)
5. University of Hyderabad. Placement Guidance and Advisory Bureau. Retrieved April 2026 from <https://gadmin.uohyd.ac.in/placement-guidance-and-advisory-bureau/> (Source of PGAB organisational structure and contact information.)
6. School of Management Studies, UoH. Placement Brochure 2024–26. (Reference material on placement process and company engagement model at school level.)
7. Google LLC. Cloud Firestore documentation. <https://firebase.google.com/docs/firestore> (Datastore semantics and security-rule model.)
8. Google LLC. Firebase Authentication documentation. <https://firebase.google.com/docs/auth> (Identity, email verification, and session management.)
9. Mankar R., Mogadpally S., Kadam M., Bhosale P., Kinage P. (2023). Smart Campus Recruitment System. IJAR SCT, 3(4), 131–142. (Related work — compared against in §2.1 for scope positioning.)

2. Overall Description

2.1 Product Perspective

The UoH PMS is a new, standalone web application developed specifically for the University of Hyderabad. It is not a derivative of, nor an extension to, any existing UoH system. It is, however, designed so that it could later be integrated with the university's academic records system, the Ernet email directory, and the IGM library services through well-defined API boundaries; such integrations are explicitly out of scope for the current release.

Compared to the general-purpose campus recruitment systems described in the academic literature (for example the Smart Campus Recruitment System by Mankar et al., 2023, Ref. [9]), the UoH PMS is deliberately narrower in feature ambition and deeper in institutional fit: it implements UoH-specific policy such as the PGAB self-declaration and blacklisting rules, it respects the university's school-department organisational structure, and it does not attempt to subsume features (online assessments, interview video rooms) that are already handled adequately by third-party tools.

2.1.1 System Context

The system interacts with five external actors and three external systems:

External Entity	Nature of Interaction
Student	Registers, maintains profile, signs the PGAB self-declaration, applies to jobs, receives notifications.
Faculty (Placement Coordinator)	Reviews and endorses applicants from their school, posts school-level notices.
TPO	Approves companies and postings, schedules drives, maintains blacklist, reviews analytics.
Recruiter	Self-registers as a company, posts jobs, reviews applications, updates application status.
Administrator	Seeds faculty and TPO accounts, manages system configuration, oversees audit log.
Firebase Authentication (external)	Performs identity verification, email-link verification, OTP generation, session token issuance.
Firestore (external)	Persists all structured application data and enforces server-side security rules.
Email Gateway (external SMTP / Firebase-triggered)	Delivers transactional email, including OTPs and status notifications.

2.1.2 System Architecture Summary

The system adopts a three-tier architecture consistent with the patterns described in §3.4 of the prescribed textbook:

- **Presentation tier** — a React 18 single-page application built with Vite and styled with Tailwind CSS, responsive from 320px (mobile) to desktop widths.
- **Application tier** — a stateless Node.js 18+ / Express 4 REST API that performs authorisation, business-rule enforcement, and orchestration; it verifies Firebase ID tokens via the Firebase Admin SDK and never holds a session of its own.
- **Data tier** — Cloud Firestore as the sole datastore, with security rules providing a defence-in-depth authorisation layer beneath the API. Firebase Authentication provides user identity.

The rationale for choosing Firestore over a relational database is discussed in the accompanying Design Document; for the purposes of this SRS, Firestore is a committed constraint (see §3.6).

2.2 Product Functions

At the highest level, the system supports nine functional capability groups. Each is decomposed into specific requirements in §3.3.

#	Capability Group	Representative Functions
F1	Identity & Access	Self-registration (Student, Company), admin-provisioned accounts (Faculty, TPO), email verification, OTP login, RBAC.
F2	Student Profile	Academic details, skills, resume upload, PGAB self-declaration e-signature.
F3	Company & Job	Company registration, job posting with eligibility rules, job approval workflow.
F4	Application Processing	Eligibility check, application submission, duplicate prevention, status tracking.
F5	Faculty Endorsement	School-level applicant review, endorsement with comments.
F6	Scheduling	Placement drive creation, interview slot booking, slot capacity enforcement.
F7	Notifications	Transactional email for registration, approvals, status changes, interview reminders.
F8	Analytics & Reports	University-wide and per-school placement dashboards, CSV/PDF export.
F9	Compliance & Audit	Self-declaration record, blacklist lifecycle, immutable audit log.

2.3 User Classes and Characteristics

The system is designed for five distinct user classes. Each has a separate role assignment, a separate set of permitted operations, and distinct usability needs.

2.3.1 Student

Volume: approximately 5,000 active students across 12 schools. **Technical proficiency:** high; predominantly aged 20–28, postgraduate and doctoral learners comfortable with web applications. **Primary tasks:** complete profile, sign self-declaration, browse eligible jobs, apply, track status, attend scheduled drives. **Expectations:** mobile-friendly interface, transparent eligibility logic, prompt email notifications, ability to see why a posting was or was not shown to them.

2.3.2 Faculty (School Placement Coordinator)

Volume: at most 12 accounts — one per School. **Technical proficiency:** moderate to high. **Primary tasks:** view their school's applicant pool, endorse candidates, post school-level notices, view school-scoped reports. **Expectations:** data is scoped to their school only; endorsements are visible to both the student and the TPO; endorsement does not bypass eligibility rules.

2.3.3 Training & Placement Officer (TPO)

Volume: exactly one active TPO account at a time, representing the central PGAB. **Technical proficiency:** moderate; functional rather than operational computer literacy. **Primary tasks:** approve new companies, approve or reject job postings, schedule placement drives, administer the blacklist (add, expire, revoke), review university-wide analytics, export reports. **Expectations:** a workbench that surfaces pending approvals first; confident audit trail of every privileged action.

2.3.4 Company Recruiter

Volume: tens to low hundreds of distinct recruiter accounts per placement season. **Technical proficiency:** variable; the interface assumes no placement-domain expertise. **Primary tasks:** register a company, post jobs once approved, view applicants, mark shortlists and outcomes. **Expectations:** clear feedback on approval status; confidential handling of candidate information; the ability to withdraw a posting.

2.3.5 Administrator

Volume: one primary and at most one secondary Administrator account, seeded at installation. **Technical proficiency:** high; typically a member of the development or operations team. **Primary tasks:** provision Faculty and TPO accounts, manage system configuration (schools, departments, notification templates), inspect the audit log. **Expectations:** no self-registration path; every Administrator action is recorded in the audit log.

2.4 Operating Environment

2.4.1 Client environment

- Modern evergreen browsers: latest two major versions of Chrome, Firefox, Edge, and Safari. Internet Explorer and legacy Edge are not supported.

- Minimum viewport width: 320 CSS pixels (mobile portrait). The interface must remain usable without horizontal scrolling at this width.
- JavaScript is required. Users with JavaScript disabled will see a graceful "please enable JavaScript" message.
- Network: the system must remain usable on consumer broadband (≥ 1 Mbps) and on mobile data; no specific corporate-network configuration is assumed.

2.4.2 Server environment

- The backend runs on Node.js 18 LTS or newer, deployed as a container image to a Linux-based host (Google Cloud Run, AWS App Runner, or comparable).
- Cloud Firestore is consumed as a managed service; no self-hosted database is used.
- Firebase Authentication is consumed as a managed identity service.
- Email delivery is through a managed SMTP gateway or Firebase-triggered cloud functions; specific provider choice is a design concern.

2.5 Design and Implementation Constraints

10. Datastore: Cloud Firestore is the sole persistence mechanism. No relational database (PostgreSQL, MySQL) is used. This is a firm constraint driven by the existing Sprint 1 implementation and operational simplicity; every requirement in §3 must be satisfiable within the Firestore model.
11. Authentication: Firebase Authentication is the sole identity provider. The system will not implement its own password storage.
12. Frontend stack: React 18, Vite, Tailwind CSS. Component libraries beyond shadcn/ui patterns are not permitted without architectural review.
13. Backend stack: Node.js 18+, Express 4, Firebase Admin SDK. The API is strictly stateless.
14. Institutional scope: Student registration is restricted to users whose email address ends in @uohyd.ac.in.
15. Policy: The system must enforce the PGAB Self-Declaration (Ref. [3]); no Student may submit a job application before signing a current-version declaration.
16. Accessibility: the interface must meet WCAG 2.1 AA at minimum.
17. Compliance: the system processes personally identifiable information under the Indian Digital Personal Data Protection Act, 2023, and the UoH privacy policy; see §3.7.2.

2.6 Assumptions and Dependencies

- The university will issue an @uohyd.ac.in email address to every enrolled student before they attempt to register.

- Faculty Placement Coordinators will be nominated by each School and their details communicated to the TPO, who will request Administrator provisioning.
- Firebase and the chosen email provider maintain their advertised service levels; the system's availability target in §3.1 assumes these upstream services are at least equally available.
- The PGAB Self-Declaration text is provided by the TPO's office and may be revised version-by-version; the system stores the text of the declaration the student signed, not just a flag.
- Students will keep their profile data current during the placement season; stale data that causes ineligibility is the student's responsibility.

3. Specific Requirements

This section enumerates every requirement the system must satisfy. Functional requirements are identified as FR-xx and are grouped by capability area. Non-functional requirements appear in §3.1 (performance), §3.7 (quality attributes) and §3.6 (constraints). Each FR is written so that a single test case can confirm or disconfirm it.

3.1 Performance Requirements

3.1.1 Response time

- PR-1. Under normal load, 95% of API responses for read operations (listing jobs, loading a profile, loading a dashboard) must complete within 2 seconds measured at the API boundary.
- PR-2. Under normal load, 95% of API responses for write operations (submitting an application, posting a job, updating a profile) must complete within 3 seconds at the API boundary.
- PR-3. First contentful paint of any authenticated page on a standard consumer broadband connection (≥ 10 Mbps, mid-tier laptop) must occur within 3 seconds.
- PR-4. OTP email delivery must begin within 30 seconds of request; the email must arrive at the recipient's mailbox within 2 minutes under normal conditions.

3.1.2 Scale

The system is sized for the UoH population and placement season pattern rather than for hypothetical internet scale. Design targets:

Metric	Expected Steady State	Peak Target
Registered Students	5,000	7,500
Active Students in a placement season	1,200	2,000
Registered Companies	300	600
Concurrent users during a drive day	200	500
Job postings per season	400	800
Applications per season	15,000	30,000
Notifications sent per day (peak)	1,000	5,000

Horizontal scaling on the application tier and Firestore's inherent scale properties are expected to absorb peak load without architectural changes.

3.1.3 Availability

- PR-5. Target monthly availability during an active placement season (August to May): 99.5% excluding scheduled maintenance windows announced at least 48 hours in advance.
- PR-6. Target monthly availability during the off-season (June to July): 99.0%.

- PR-7. No single deployment of the application tier may exceed 60 seconds of downtime; rolling deployments are required.

3.1.4 Capacity limits

- PR-8. Resume uploads are limited to PDF format, 5 MB per file.
- PR-9. A Student may store a maximum of 3 resume versions at any time; additional uploads replace the oldest.
- PR-10. A Company may have up to 20 concurrently-active job postings per season.

3.2 External Interface Requirements

3.2.1 User Interfaces

- UI-1. The frontend is a single-page application. Deep-linking must work: a URL copied from the address bar, when reopened by an authenticated user with sufficient permission, must navigate to the same view.
- UI-2. Each role has a distinct landing dashboard after login. Navigation elements permitted to a role are visible; those not permitted are hidden rather than displayed-and-disabled.
- UI-3. Every form input has a visible label (not placeholder-only), inline validation on blur, and a clear error state.
- UI-4. Long lists (jobs, applications, candidates) must support sort, filter, and pagination with a default page size of 20.
- UI-5. The visual design follows the University of Hyderabad brand identity: primary colour is UoH Maroon (hex #7A0019) derived from the official UoH emblem; typography uses a serif display face for headings and a neutral sans-serif for body.
- UI-6. The interface conforms to WCAG 2.1 Level AA: minimum contrast 4.5:1 for body text, keyboard operable, ARIA landmarks on every page.

3.2.2 Hardware Interfaces

The system has no direct hardware dependencies beyond the commodity cloud-compute on which it is hosted. It does not interact with biometric devices, printers, or physical proctoring hardware.

3.2.3 Software Interfaces

Interface	Direction	Purpose
Firebase Authentication	Client & Server → Firebase	User registration, email verification, sign-in, token issuance, token verification.
Cloud Firestore	Server → Firestore (Admin SDK)	All persistent reads and writes. Client SDK is not used for data writes in this system; all writes are mediated by the API.

Interface	Direction	Purpose
Email Gateway	Server → SMTP/provider API	Delivery of OTP, registration confirmation, approval notices, application-status updates, interview reminders.
Web Browser	Client ↔ User	Rendering of the React SPA.

3.2.4 Communications Interfaces

- CI-1. All client-server communication uses HTTPS with TLS 1.2 or later. Plain-HTTP endpoints must not exist in production.
- CI-2. The API uses JSON request and response bodies with the Content-Type application/json; charset=utf-8.
- CI-3. Authenticated requests carry a Firebase ID token in the Authorization header as a Bearer token.
- CI-4. Cross-Origin Resource Sharing is restricted to the production frontend origin and the local development origin; no wildcard origin is permitted.

3.3 Functional Requirements

Functional requirements are grouped by capability area (F1 – F9, as introduced in §2.2). Each requirement is stated at a level a test case can verify directly. Cross-references between requirements are by identifier.

3.3.1 F1 — Identity & Access Management

Student self-registration

- FR-1.1. A visitor with an email address ending in @uohyd.ac.in may open the self-registration form and submit their full name, email, password (meeting the policy in FR-1.14), School, Department, programme, and year of graduation.
- FR-1.2. The system must reject any registration whose email does not end in @uohyd.ac.in with a message explaining the restriction.
- FR-1.3. Upon submission, the system creates a Firebase user with status 'Unverified', sends a verification link to the email address, and presents an instruction screen.
- FR-1.4. Until the verification link has been clicked, the account cannot sign in.
- FR-1.5. On first sign-in after verification, the system must require the Student to electronically sign the current PGAB Self-Declaration (FR-2.7) before permitting any other action.

Company self-registration

- FR-1.6. A visitor may submit a company registration with company name, official website, HR contact name, HR contact email (any domain), HR phone number, and a short organisation description. Any valid email domain is accepted.

- FR-1.7. Upon submission, the system creates an account in 'Pending Approval' status and notifies the Administrator and TPO.
- FR-1.8. A pending company may not sign in or post jobs; attempted sign-in returns a clear "Your company is awaiting approval" message.
- FR-1.9. When the TPO or Administrator approves the company, the system changes the status to 'Active', sends a welcome email to the HR contact, and permits sign-in.
- FR-1.10. When the TPO or Administrator rejects a company, the system sets status to 'Rejected', sends an explanatory email, and prohibits sign-in.

Faculty and TPO provisioning

- FR-1.11. Faculty accounts can only be created by an Administrator action. The Administrator specifies the Faculty member's name, @uohyd.ac.in email, and the single School to which they are assigned as Placement Coordinator.
- FR-1.12. There may be at most one active Faculty account per School at any time.
- FR-1.13. The TPO account is created by an Administrator action. There is exactly one active TPO account in the system at any time; provisioning a new TPO deactivates the previous one.

Authentication and sessions

- FR-1.14. Passwords must be at least 8 characters and must contain at least one uppercase letter, one lowercase letter, and one digit. Rejection messages must specify the rule that was violated.
- FR-1.15. On every successful password verification, the system issues a six-digit numeric OTP to the user's registered email, valid for 10 minutes. The user must enter this OTP before a session is established.
- FR-1.16. An OTP may be entered at most three times; on the fourth incorrect entry the OTP is invalidated and the user must restart sign-in.
- FR-1.17. Session tokens issued by Firebase are valid for 60 minutes and must be refreshed silently by the client.
- FR-1.18. A user may request "forgot password" at any time; the system sends a password-reset link, valid for 30 minutes, to the registered email.

Authorisation

- FR-1.19. Every API endpoint performs role-based authorisation. A request from a role without permission for that endpoint returns HTTP 403 with a generic "Insufficient permission" message and is recorded in the audit log.
- FR-1.20. The Administrator role cannot be assigned through any self-service or in-app action; Administrator accounts are provisioned only by infrastructure-level seeding.

3.3.2 F2 — Student Profile & Self-Declaration

Profile management

- FR-2.1. A Student must be able to view their own profile and edit: personal information (photo, phone), academic information (School, Department, programme, year of joining, expected year of graduation, current CGPA, backlog count), and skill list.
- FR-2.2. CGPA must be accepted only as a decimal number between 0.0 and 10.0 inclusive, with up to two decimal places.
- FR-2.3. A Student may upload a PDF resume up to 5 MB. The system stores up to three versions; the most recent version is flagged as the default used for one-click applications.
- FR-2.4. Changes to CGPA, School, Department, or backlog count must be recorded in the Student's profile-change history. This history is visible to the Student, to their School's Faculty, and to the TPO.
- FR-2.5. A Student may not delete their own profile. Account deactivation is an Administrator action (FR-9.6).

PGAB Self-Declaration

- FR-2.6. The system stores the current text of the PGAB Self-Declaration as a versioned document. Each revision is assigned a sequential version identifier.
- FR-2.7. Before a Student may submit their first job application, or whenever the declaration version has been incremented since the Student's last signature, the system must present the current declaration text and require the Student to check an explicit consent box and enter their full name as a typed e-signature.
- FR-2.8. The system records the Student ID, the declaration version signed, the full text of that version, the timestamp of signature, and the IP address from which it was signed.
- FR-2.9. A Student may download a PDF copy of any declaration they have signed.
- FR-2.10. The system must not permit the creation of a job application by a Student who has not signed the current-version declaration.

3.3.3 F3 — Company and Job Management

- FR-3.1. An approved Company account may create a job posting with: position title, job description (rich text, up to 10,000 characters), location, employment type (full-time, internship, PPO), CTC (numeric, Indian rupees per annum), application deadline, and eligibility criteria.
- FR-3.2. Eligibility criteria include: eligible Schools (one or more), eligible Departments (one or more, filtered to the selected Schools), minimum CGPA, maximum allowed backlogs, eligible programmes, eligible years of graduation.
- FR-3.3. A newly posted job enters status 'Pending Approval' and is not visible to Students.
- FR-3.4. The TPO may approve or reject a pending job. On approval, the job becomes visible to eligible Students; on rejection, the Company is notified with the reason.

- FR-3.5. A Company may withdraw one of its own jobs at any time; withdrawal sets status to 'Withdrawn', hides the job from Students, and notifies any Students who had applied.
- FR-3.6. The TPO may close a job after its deadline; closed jobs accept no new applications.

3.3.4 F4 — Application Processing

- FR-4.1. The "Eligible Jobs" listing for a Student must include only postings that: (a) are in status 'Open', (b) have not passed their deadline, (c) match the Student's School and Department against the posting's eligibility criteria, (d) have a minimum-CGPA constraint $\leq$ the Student's current CGPA, (e) have a maximum-backlog constraint $\geq$ the Student's current backlog count, (f) match the Student's programme and expected graduation year, and (g) do not have the Student on the active blacklist for the configured duration.
- FR-4.2. A Student may apply to an eligible posting by confirming a single action. The system records the application with: applicant ID, job ID, default-resume version, timestamp, initial status 'Applied'.
- FR-4.3. A Student may not apply to the same job more than once; the application button is disabled and a clear indication is shown for already-applied jobs.
- FR-4.4. A Student may withdraw their own application while the job is still open; withdrawal sets status to 'Withdrawn (by Student)' and is final — re-application is not permitted.
- FR-4.5. The application-status lifecycle is: Applied $\rightarrow$ Shortlisted $\rightarrow$ Interview Scheduled $\rightarrow$ Interviewed $\rightarrow$ {Selected | Rejected | Waitlisted}. A Withdrawn status may be reached from any state before Selected.
- FR-4.6. Only the Company that owns the job, the TPO, and the Administrator may change an application's status. A status change generates a notification to the Student.

3.3.5 F5 — Faculty Endorsement

- FR-5.1. A Faculty member may view the list of applicants to any job, restricted to applicants whose School matches their assigned School.
- FR-5.2. A Faculty member may record an endorsement for any applicant in their School for any specific job: endorsement is a Boolean with an optional comment up to 500 characters.
- FR-5.3. Endorsement is visible to the Student, to the Company, and to the TPO. An endorsement is informational; it does not override eligibility rules and it does not by itself shortlist or select the candidate.
- FR-5.4. A Faculty member may post school-level notices (title, body, optional attachment) visible to Students of their own School only.

3.3.6 F6 — Placement Drive Scheduling

- FR-6.1. The TPO may create a Placement Drive with: title, associated Company, associated Job(s), date, start time, duration, venue, total slot capacity.

- FR-6.2. The TPO may break the Drive into one or more Interview Slots, each with its own time window and capacity.
- FR-6.3. Students whose application for a drive's job has status 'Shortlisted' may book a slot. Booking is accepted only if the slot has remaining capacity.
- FR-6.4. A Student may book at most one slot per Drive; the system must not permit overlapping bookings across Drives on the same day.
- FR-6.5. A Student may reschedule to a different slot within the same Drive up to 24 hours before their original slot's start time, subject to capacity.
- FR-6.6. The system sends an email reminder to every Student with a booked slot 24 hours before the slot start time.
- FR-6.7. A Student who does not attend a booked slot, without reschedule, is flagged as a no-show. The TPO may escalate a no-show to a blacklist action (see FR-9.1).

3.3.7 F7 — Notifications

- FR-7.1. The system must send an email notification for each of the following events: account verification required, OTP delivery, company approval/rejection, job approval/rejection, job posted (to eligible Students — optional opt-in), application submitted confirmation, application status change, interview slot booked, interview reminder 24h before, blacklist applied, blacklist lifted.
- FR-7.2. Every email notification must include the sender "UoH Placement Management System", a human-readable subject line, the recipient's name, the reason the email was sent, and a footer identifying the system and the TPO's office.
- FR-7.3. The TPO may send a broadcast email to a filtered Student cohort (by School, Department, programme, year); the recipient count is shown before sending, and the message is recorded in the audit log.
- FR-7.4. In-app notifications appear in a notification tray on every page; unread notifications show a counter.

3.3.8 F8 — Analytics and Reporting

- FR-8.1. The TPO dashboard shows, for the current placement season: number of registered Students, active Students, registered Companies, open jobs, total applications, Students with at least one active application, Students Selected, total offers, average CTC, median CTC, highest CTC.
- FR-8.2. The TPO may drill into any metric to see its underlying list (e.g., the 142 Selected Students → the list of 142 rows).
- FR-8.3. A Faculty member's dashboard shows the same metrics but scoped to their own School only.

- FR-8.4. The TPO may export placement data in CSV or PDF format, with filters for date range, School, Department, Company, job type.
- FR-8.5. Reports are always generated on-demand from current data; there is no pre-computed overnight snapshot that could drift from reality.

3.3.9 F9 — Compliance and Audit

- FR-9.1. The TPO may place a Student on the blacklist, recording: Student ID, reason (enumerated: no-show, offer-rejection-after-acceptance, misconduct, other-with-comment), severity (configurable: 1–5 future drives), start date, and expected end date.
- FR-9.2. A blacklisted Student sees a banner on their dashboard explaining the reason, the affected drives, and the expected end date. The banner is also shown on the jobs listing.
- FR-9.3. A blacklisted Student's eligibility rules (FR-4.1) automatically exclude them from the configured number of subsequent drives.
- FR-9.4. The TPO may lift a blacklist entry early, recording the reason for lifting.
- FR-9.5. Every privileged action is recorded in the audit log with: actor ID, actor role, action type (approve-company, reject-job, create-drive, blacklist-add, blacklist-lift, role-change, broadcast-email, configuration-change), target entity, timestamp, source IP address.
- FR-9.6. Account deactivation (for any role) is an Administrator action; the system does not hard-delete an account, and the audit log and historical applications remain intact.
- FR-9.7. Audit log entries are append-only. No user-facing action can modify or delete an entry.

3.4 Analysis Classes / Data Dictionary

This section identifies the persistent data entities the system manages. Each entity is described in terms of its responsibilities, key attributes, and relationships. These entities map directly to Firestore collections in §3.5. This view is logical; the physical Firestore schema — with subcollections, denormalisation choices, and security rules — is specified in the Design Document.

One architectural note is important here. In the previous version of this SRS, the User hierarchy was modelled with Student, Faculty, Administrator, and Company as subclasses of User (inheritance). In this revision, the User entity holds a single role attribute and role-specific attributes are carried by associated entity documents. This change aligns the data model with the Sprint 1 implementation, where user role is a string on the user document rather than a class hierarchy. It also simplifies Firestore security rules, which cannot express inheritance. The reasoning follows the cohesion and coupling discussion of §3.2 of the prescribed textbook and is expanded in the Design Document.

3.4.1 Core entities

User

The identity record. Every sign-in binds to exactly one User.

Attribute	Type	Notes
userId	String (UID)	Firebase Authentication UID. Primary key.
email	String	Verified email address; unique.
fullName	String	Display name.
role	Enum	One of: student, faculty, tpo, company, admin.
status	Enum	active, pending-approval, unverified, suspended, deactivated.
createdAt	Timestamp	Account creation time.
lastLoginAt	Timestamp	Most recent successful OTP step completion.

StudentProfile

One-to-one with a User whose role is student.

Attribute	Type	Notes
userId	String (FK)	Matches the User.
rollNumber	String	University roll / enrolment number.
schoolId	String (FK)	Reference to School.
departmentId	String (FK)	Reference to Department within that School.
programme	String	e.g. M.Tech, MCA, Ph.D.

Attribute	Type	Notes
joiningYear	Integer	
graduationYear	Integer	
cgpa	Decimal(0.0–10.0, 2 places)	Current CGPA.
backlogs	Integer	Current active backlog count.
skills	Array<String>	Self-reported skill tags.
resumes	Array<ResumeRef>	Up to 3 versions; one flagged default.
phoneNumber	String	With country code.
profileComplete	Boolean	True once mandatory fields are filled.

FacultyProfile

Attribute	Type	Notes
userId	String (FK)	
schoolId	String (FK)	The single School this Placement Coordinator represents.
designation	String	Free-text: Professor, Associate Professor, etc.
phoneNumber	String	

Company

Attribute	Type	Notes
companyId	String	Primary key.
primaryContactUserId	String (FK)	The Recruiter user who registered the company.
companyName	String	
website	String (URL)	
description	String	Up to 2,000 characters.
hrContact	Object	{ name, email, phone }.
industry	String	Optional taxonomy value.
status	Enum	pending-approval, active, rejected, suspended.
approvedAt	Timestamp	Null until approved.
approvedBy	String (FK)	User ID of approver.

Job

Attribute	Type	Notes
jobId	String	Primary key.
companyId	String (FK)	
title	String	
description	String	Rich text, up to 10,000 characters.
location	String	
employmentType	Enum	full-time, internship, ppo.
ctcPerAnnum	Decimal (INR)	
deadline	Timestamp	
eligibility	Object	{ schools[], departments[], minCgpa, maxBacklogs, programmes[], graduationYears[] }.
status	Enum	pending-approval, open, closed, rejected, withdrawn.
postedAt	Timestamp	
approvedAt	Timestamp	

Application

Attribute	Type	Notes
applicationId	String	Primary key.
jobId	String (FK)	
studentId	String (FK)	
resumeVersionId	String (FK)	The resume used at application time.
status	Enum	applied, shortlisted, interview-scheduled, interviewed, selected, rejected, waitlisted, withdrawn-student, withdrawn-system.
statusHistory	Array<StatusChange>	{ status, changedAt, changedBy, reason? }.
appliedAt	Timestamp	
facultyEndorsement	Object?	{ facultyId, endorsed, comment, endorsedAt }.

Drive, InterviewSlot, SlotBooking

These three entities together model the scheduling domain. A Drive is scheduled by the TPO; it comprises one or more InterviewSlots; a SlotBooking is a Student's reservation of one slot.

Entity	Key attributes
Drive	driveId, jobIds[], date, startTime, durationMinutes, venue, totalCapacity, status
InterviewSlot	slotId, driveId, startTime, endTime, capacity, bookedCount
SlotBooking	bookingId, slotId, studentId, applicationId, bookedAt, attendanceStatus (booked, attended, no-show, rescheduled)

Declaration (new)

Represents a specific version of the PGAB Self-Declaration text, and each Student's signing of it.

Entity	Key attributes
DeclarationVersion	versionId, version, text, effectiveFrom, publishedBy
DeclarationSignature	signatureId, studentId, versionId, signedAt, ipAddress, typedName, textSnapshotHash

Blacklist (new)

Attribute	Type	Notes
blacklistId	String	
studentId	String (FK)	
reason	Enum + comment	no-show, offer-rejection-after-acceptance, misconduct, other.
severity	Integer	Number of future drives affected (1–5).
startDate	Timestamp	
endDate	Timestamp	
status	Enum	active, expired, lifted.
createdBy	String (FK)	TPO user ID.
liftedBy / liftedReason	String / String?	When lifted.

AuditLog (new)

Attribute	Type	Notes
entryId	String	
actorUserId	String (FK)	
actorRole	Enum	

Attribute	Type	Notes
actionType	Enum	approve-company, reject-company, approve-job, reject-job, create-drive, blacklist-add, blacklist-lift, role-change, broadcast-email, config-change, etc.
targetType / targetId	Enum / String	e.g. company / companyId.
payloadSummary	String	Short human-readable summary (≤ 500 chars).
ipAddress	String	
createdAt	Timestamp	

School, Department (reference data)

Seeded from the authoritative UoH list of 12 Schools and 42 Departments (Ref. [4]).

Entity	Key attributes
School	schoolId, name, shortCode
Department	departmentId, schoolId, name, programmes[]

3.4.2 Data Dictionary conventions

- All identifiers are opaque document IDs; no business meaning is encoded in an identifier.
- All timestamps are stored in UTC and rendered in Asia/Kolkata on the client.
- All monetary values are stored as decimal numbers in Indian Rupees (INR).
- All enumerations are validated server-side on every write.

3.5 Logical Database Requirements

The system uses Cloud Firestore as its sole persistence mechanism. This section states the logical data requirements; the physical collection structure, indices, and security rules are developed in the Design Document.

3.5.1 Firestore model — logical view

Firestore is a document database organised in collections. The logical mapping from the entities in §3.4 to Firestore collections is:

Collection	Document shape	Notes
users	User	Keyed by Firebase UID.
studentProfiles	StudentProfile	Keyed by userId.
facultyProfiles	FacultyProfile	Keyed by userId.
companies	Company	One per registered organisation.
jobs	Job	
applications	Application	
drives	Drive	
interviewSlots	InterviewSlot	
slotBookings	SlotBooking	
declarationVersions	DeclarationVersion	Small; typically ≤ 10 entries in a system lifetime.
declarationSignatures	DeclarationSignature	Keyed by signatureId; indexed on studentId.
blacklists	Blacklist	
auditLogs	AuditLog	Append-only; no client write access.
schools	School	Seeded reference data.
departments	Department	Seeded reference data.

3.5.2 Integrity requirements

- DB-1. Every foreign-key attribute (userId, schoolId, departmentId, jobId, studentId, driveId, etc.) must refer to an existing document at the time of write; missing references must cause the write to fail.
- DB-2. Unique constraints: a User is unique by email; a StudentProfile is unique by userId; a FacultyProfile is unique by userId and also by schoolId (at most one active Faculty per school).
- DB-3. All write operations must pass through the API; direct writes from untrusted clients must be blocked by Firestore security rules.

- DB-4. The auditLogs collection is write-once: no update or delete operations are permitted from any client, including the API using the Admin SDK, except through a dedicated retention-policy tool run under Administrator control.
- DB-5. Application writes that depend on a consistent eligibility check (for example, applying to a job) must be performed inside a Firestore transaction so that concurrent changes to the Student's CGPA or the Job's status cannot produce an inconsistent application.
- DB-6. Every document is timestamped with createdAt on insert and updatedAt on every modification.

3.5.3 Backup and retention

- DB-7. Firestore exports must be scheduled nightly at off-peak hours and retained for at least 90 days.
- DB-8. Placement season records (jobs, applications, drives) must be retained for at least 5 years after the end of the relevant academic year to support NIRF and IoE reporting.
- DB-9. Self-Declaration signatures and audit logs must be retained for at least 7 years.
- DB-10. Deactivated User accounts are retained — never hard-deleted — to preserve referential integrity of historical records.

3.6 Design Constraints

The following constraints bind the design. They are stated here, in the SRS, because they influence the observable behaviour and testable characteristics of the system, not merely its internal construction.

18. DC-1. Firestore is the sole persistence technology. No relational database appears in production.
19. DC-2. Firebase Authentication is the sole identity provider. The system does not store or hash passwords itself.
20. DC-3. The backend is stateless. No session state is held on the server between requests.
21. DC-4. The frontend is delivered as a single-page application from a static-hosted bundle; server-side rendering is not used.
22. DC-5. All secrets (service-account keys, API keys, SMTP credentials) are held in environment variables or a managed secret store; no secret appears in source control or in a client-side bundle.
23. DC-6. Source control is Git; code review is mandatory on all merges to the main branch.
24. DC-7. The REST API follows a noun-based URL scheme (for example `/api/jobs`, `/api/jobs/{id}`, `/api/jobs/{id}/applications`). HTTP verbs carry the action (GET, POST, PATCH, DELETE).
25. DC-8. The code base must carry unit tests for all authorisation middleware, authentication controllers, and eligibility logic; test coverage of these modules must be $\geq 80\%$ line coverage.

3.7 Software System Attributes (Quality Requirements)

The quality attributes below are non-functional but individually testable. The list uses the categories from the prescribed textbook §2.6 and from IEEE 29148.

3.7.1 Reliability

- QA-R1. The system must not lose a successfully acknowledged write under any single-instance failure. Firestore's own durability guarantees are relied upon for this property.
- QA-R2. After a deployment failure, the system must roll back to the previous known-good version automatically within 5 minutes.
- QA-R3. All error responses must be logged centrally; any 5xx response rate exceeding 1% over a 15-minute window must trigger an alert.

3.7.2 Security

- QA-S1. All data in transit uses TLS 1.2 or later.
- QA-S2. Firebase Authentication enforces password storage off-server, OAuth-grade token handling, and email-link verification.
- QA-S3. The API is hardened with helmet HTTP headers, a strict CORS policy, and request rate limiting at 100 requests per minute per IP for authenticated endpoints and 20 requests per minute for unauthenticated ones.

- QA-S4. Administrative privilege escalation — such as assigning the TPO role — is possible only through an Administrator-authenticated session against a dedicated endpoint, and is logged in the audit log (F9).
- QA-S5. The system complies with the Indian Digital Personal Data Protection Act, 2023: processing of Student PII is limited to placement purposes, and Students may request an export of their personal data in machine-readable form from the TPO.

3.7.3 Maintainability

- QA-M1. The code base is modular: the presentation tier, API tier, and data-access layer are separable and independently testable.
- QA-M2. All REST endpoints are documented using OpenAPI 3.0 and the specification is versioned with the code.
- QA-M3. The system logs structured JSON on every API call: request method, path, user ID, role, status code, latency.

3.7.4 Portability

- QA-P1. The backend runs in a container image and does not depend on any host-specific binary; it can be re-deployed from Google Cloud Run to AWS App Runner or to Kubernetes without code changes.
- QA-P2. The frontend builds to a static asset bundle and can be served by any static host (Firebase Hosting, Cloudflare Pages, Vercel, Netlify, S3+CloudFront).

3.7.5 Usability

- QA-U1. A first-time Student must be able to complete registration, email verification, self-declaration, and first application with no external guidance, in under 10 minutes.
- QA-U2. The interface uses plain English; error messages name the rule that was violated rather than technical error codes.
- QA-U3. The interface supports full keyboard navigation; every interactive element has a visible focus indicator.

3.7.6 Accessibility

- QA-A1. The interface meets WCAG 2.1 Level AA success criteria on every page.
- QA-A2. All informational icons have an accessible name or an adjacent text label.
- QA-A3. Form errors are announced to assistive technology via aria-live polite regions.

4. Support Information

4.1 Traceability

The Software Engineering textbook in use (Ref. [1]) emphasises requirements traceability. Every functional requirement in this SRS has an identifier of the form FR-x.y. Every such identifier is expected to appear in (a) at least one use case or user story in the Design Document, (b) at least one test case in the Test Plan, and (c) at least one acceptance check in the user-acceptance script. Non-functional requirements (PR-, DB-, DC-, QA-) are similarly traced.

A consolidated requirements-to-test traceability matrix is maintained in the Test Plan document. The SRS and the Test Plan are expected to be cross-reviewed on every SRS revision.

4.2 Process Model

This project follows a Scrum development model (Unit 1 §1.11 of Ref. [1]). Requirements are grouped into five two-week sprints:

Sprint	Scope	Corresponding SRS sections
1	Identity & access foundation	F1 (FR-1.x), PR-4, QA-S1–S3, DC-1–5
2	Student profile, Self-Declaration, Companies, Jobs, Applications	F2 (FR-2.x), F3 (FR-3.x), F4 (FR-4.x), DB-5
3	Scheduling, Faculty endorsement, Notifications	F5 (FR-5.x), F6 (FR-6.x), F7 (FR-7.x)
4	Analytics, Blacklist, Audit Log, Admin tooling	F8 (FR-8.x), F9 (FR-9.x), QA-M2, QA-M3
5	Hardening, usability polish, test completion, documentation	QA-U1–U3, QA-A1–A3, Test Plan, Design Document

Scope changes during a sprint are restricted to the Product Backlog and never to the in-flight Sprint Backlog; this rule is consistent with §1.11.2 of the textbook.

4.3 Open Issues and Future Work

- A native mobile application is not in scope; the responsive web frontend is used on mobile at launch.
- Online proctored assessments are deferred to a post-MVP release.
- Integration with the UoH academic records system to auto-verify CGPA and backlog data is a candidate for a future release; a formal API agreement with the Controller of Examinations would be required.
- SMS notifications through Twilio or a comparable gateway are planned as a hardening feature for Sprint 4+, contingent on budget approval. The current release uses email-only notification.

- Native Single Sign-On against the UoH Ernet identity provider (should one become available) would remove the need for a separate password for Students and is a candidate for future work.

4.4 Glossary of Specifically UoH Terms

Term	Meaning in this system
PGAB Self-Declaration	The PGAB candidate declaration document, version-controlled in the system, that every Student must e-sign before applying to any opportunity.
Drive	A scheduled placement event organised by the TPO on a given day for a given Company, comprising one or more interview slots.
Placement season	The academic-year window during which the majority of hiring activity takes place, typically August–May.
Blacklist	A time-boxed restriction on a Student's ability to apply to or attend future drives, imposed by the TPO under PGAB rules.
Eligibility	The set of rules, on a given Job, that determine which Students may apply — composed of School, Department, programme, graduation year, minimum CGPA, and maximum backlogs.

4.5 Approval and Baselining

This SRS is baselined on the date shown in §0 (Document Control — Revision History) as version 2.0. Subsequent changes require a new revision entry, a summary of changes, and re-review by the supervising faculty before implementation begins. Any change that materially alters the role model, the enforced policies, or the datastore requires explicit re-review.

4.6 Document End

This document ends here. Following artefacts — the Design Document (Unit 3), the Test Plan and Test Report (Unit 4), and the Project Management artefacts (Unit 5, including WBS, activity network, and risk register) — elaborate on, and are bound by, this specification.